



efrei

PARIS PANTHÉON-ASSAS UNIVERSITÉ

RAPPORT SM602

RECHERCHE

OPÉRATIONNELLE

Theo UNG
Florian LEROUX
Kellianne LAWS
Nancy THOMAS
Mael LECARPENTIER

INTRODUCTION

Dans le cadre de ce projet de recherche opérationnelle, nous avons d'abord programmé trois algorithmes phares de circulation de flot sur les graphes: Ford-Fulkerson, Push-Relabel et le Flot à coût minimal.

Une fois leur bonne exécution vérifiée sur les instances fournies, nous avons mené une étude expérimentale de leur complexité. Ces méthodes ont pour but d'acheminer un flot d'une source vers un puits tout en respectant les capacités des arêtes; pour le dernier algorithme, il s'agit en outre de minimiser le coût global.

Notre objectif était de mesurer l'évolution du temps d'exécution en fonction de la taille n du graphe, en nous concentrant sur le pire des cas. Pour ce faire, nous avons généré des graphes aléatoires et réalisé 100 mesures par valeur de n afin de tracer les nuages de points et d'en extraire les temps maximaux.

GÉNÉRATION DES GRAPHES

Pour évaluer la complexité, nous avons écrit un programme en Python qui produit, pour un entier n , des graphes orientés aléatoires. Chaque instance est codée par deux matrices carrées: l'une pour les capacités, l'autre pour les coûts.

Les matrices sont d'abord remplies de zéros, puis nous tirons au hasard $\approx \frac{N^2}{2}$ couples distincts (i, j) avec $i \neq j$.

À chaque arête retenue, nous attribuons une capacité et un coût entiers uniformes dans l'intervalle $[1, 100]$. Cette méthode fournit des graphes sans boucles, de densité moyenne ($\approx 50\%$), parfaitement adaptés à la mise à l'épreuve des algorithmes sur un large éventail de tailles et de configurations.

MESURE DES TEMPS D'EXÉCUTION

Les durées d'exécution sont relevées avec **time.process_time()** de Python, qui mesure le temps CPU consommé par le processus.

Pour chaque lancement du programme, l'horloge est lue juste avant puis juste après l'appel de l'algorithme; leur différence fournit la durée, ensuite consignée dans des fichiers .txt pour analyses ultérieures.

Les expérimentations couvrent un large éventail de tailles de graphe, avec $n \in \{10, 20, 40, 100, 400, 1000, 4000, 10000\}$. Ce choix d'échantillons espacés nous permet d'observer finement la transition entre les régimes «petit», «moyen» et «très grand», tout en restant compatible avec la limite mémoire de la machine de test. Pour chaque valeur de n , nous réalisons 100 exécutions indépendantes.

NUAGES DE POINTS & TENDANCE MOYENNE

Pour exploiter les données consignées dans les fichiers TXT, nous traçons, pour chaque algorithme, un nuage de points où :

- l'axe des x porte la taille du graphe (nombre de sommets n).
- l'axe des y indique le temps CPU mesuré à l'exécution.

La dispersion des points met en évidence une progression proche d'une courbe quadratique : à mesure que n double, la durée s'accroît approximativement par un facteur 4, ce qui suggère une complexité en $O(n^2)$.

ANALYSE DU CAS EXTRÊME

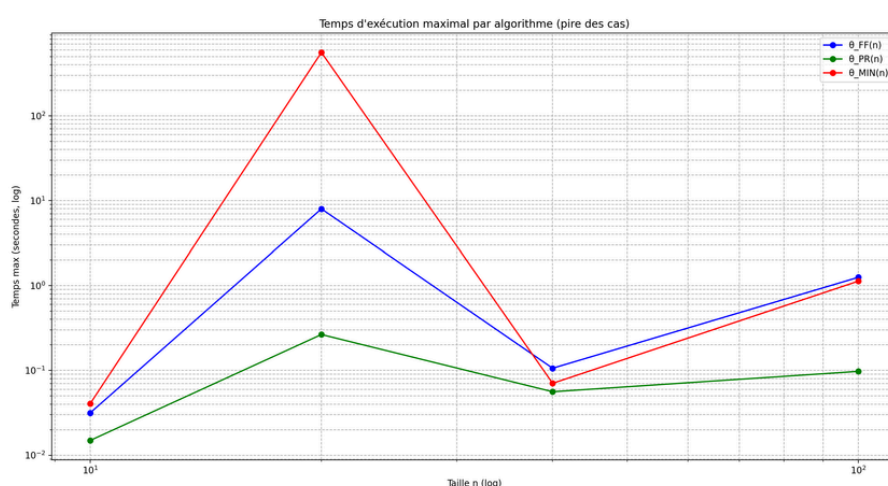
Notre implémentation de Ford-Fulkerson s'arrête dès qu'il n'existe plus aucun chemin améliorant, signe que le flot atteint sa valeur maximale.

L'algorithme Push-Relabel, quant à lui, se termine lorsque plus aucun sommet n'est actif, c'est-à-dire qu'il ne subsiste plus d'excédent à pousser.

Pour évaluer leurs performances respectives, nous calculons le rapport des temps d'exécution :

$$Rapport(n) = \frac{\theta_{FF}(n)}{\theta_{PR}(n)}$$

Nous représentons ensuite cette grandeur, obtenue pour chaque taille de graphe, à l'aide de matplotlib afin de visualiser l'avantage relatif de Push-Relabel sur Ford-Fulkerson (ou inversement) selon n :



COMPARAISON INTER-ALGORITHMES

Pour apprécier l'efficacité des trois méthodes, il suffit de confronter leurs bornes théoriques ; on raisonne en fonction du nombre de sommets $|V|$ et d'arêtes $|E|$.

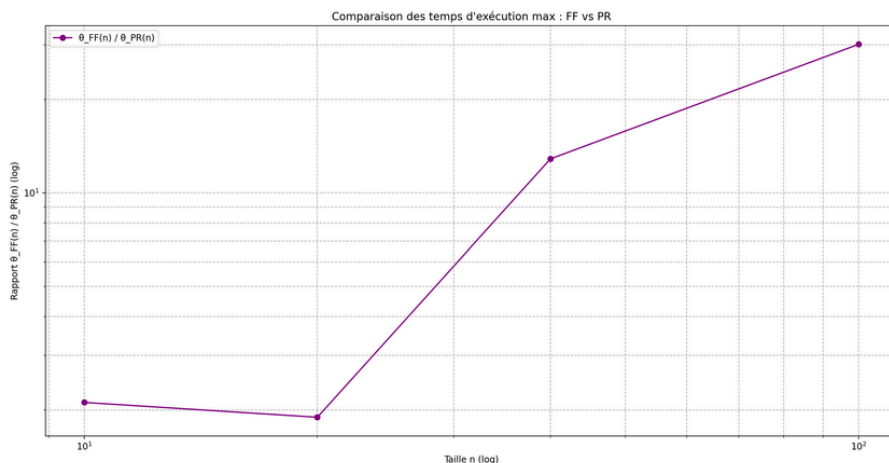
ALGORITHME	COMPLEXITÉ
FORD FULKERSON	$O(V E ^2)$
PUSH RELABEL	$O(V E)$
FLOT À COÛT MINIMAL	$O(V E ^2)$

On suppose que chaque sommet (hormis le puits t) possède au moins une arête sortante. Dans la plupart des graphes, dès qu'au moins un sommet a plus d'une arête sortante, on a :

$$|V||E|^2 \geq |V|^2|E| \text{ car } |E| \geq |V|$$

L'égalité des deux bornes n'a lieu que dans un cas très particulier: celui d'un graphe quasi arborescent, où chaque sommet (sauf la source et le puits) possède exactement une arête sortante.

En pratique, l'algorithme de Ford-Fulkerson se révèle donc supérieure à la borne l'algorithme de Push-Relabel. Donc pour résoudre un problème de flot maximal sur des graphes usuels (non arborescents), Push-Relabel est théoriquement moins coûteux en temps et constitue le choix privilégié.



CONCLUSION

Ce travail a implémenté et testé trois algorithmes de flot: Ford-Fulkerson, Push-Relabel et Flot à coût minimal sur des graphes aléatoires de tailles croissantes. Les mesures confirment la théorie: Push-Relabel surpasse Ford-Fulkerson sur les grands graphes, et l'algorithme de Flot à cout minimal reste incontournable quand un coût est associé aux arêtes. Le choix de l'algorithme doit donc se caler sur les contraintes du problème, quitte à envisager des approches hybrides pour améliorer encore les performances.